

Experiment - 3

Aim of the Experiment: Assembly language Programming of 8085 using Logical, Rotate and Jump Instructions

Theory:

1) Logical Instructions

- AND Operations: ANA (AND Register/Memory), ANI (AND Immediate) - performs bitwise AND with the accumulator
- OR Operations: ORA (OR Register/Memory), ORI (OR Immediate) - performs bitwise OR with the accumulator
- XOR Operations: XRA (XOR Register/Memory), XRI (XOR Immediate) - performs bitwise Exclusive OR with the accumulator
- Complement operations: XRA
CMA: Complement (NOT) the Accumulator
CMC: Complement Carry flag
STC: Set carry flag
- Rotate operations: shifts bits within the accumulator.
RLC: Rotate Accumulator left (D7 to D0)
RRC: Rotate Accumulator right (D0 to D7)
RAL: Rotate Accumulator Left through Carry (D7 to CY, CY to D0).
RAR: Rotate Accumulator Right through Carry

(DO to CY, CY to DF).

→ Compare operations: Compare data with the accumulator and set flags

CMP (Compare Register/Memory): Performs a subtraction (A-operation) internally, without altering the accumulator's or operand data, but setting the zero (Z) and Carry (CY) flags to indicate the result.

→ CPD (Compare Immediate Data).

2) Jump Instructions

→ Unconditional Jump

JMP (Jump): Transfers control of the specified 16 bit address regardless of any conditions, essential for infinite loops or jumping over code.

3) Conditional Jump

These check the state of the 8085's flag bits (Sign, Zero, Parity, Carry, etc.)

→ JNZ/JZ: Jump if Not Zero/Jump if Zero

→ JC/JNC: Jump if Carry/Jump if No Carry

→ JM/JP: Jump if Minus/Jump if plus

→ JPE/JPO: Jump if Parity Even/Jump if Parity odd

Problem Statement:

1) Multiplication of 2 8 bit-numbers at consecutive memory locations 2500H and

2501H by

a) Repeated addition method.

Soln: Memory

- 2500H - Multiplicand
- 2501H → Multiplier
- 2502H → Lower byte of a result
- 2503H → Upper byte of a result

Program Explanation

- LXI H, 2500 - Load HL pair with starting address of operands
- MVI M, 05 - Store first number at 2500H
- INXH - Increment HL to next memory location
- MVI M, 03 - Store second number at 2501H
- LXI H, 2500 - Reload HL with base address
- MOV B, M - Load multiplicand into register B
- INXH - Point HL to multiplier
- MOV C, M - Load multiplier into register C
- ADD B - Add multiplicand to accumulator
- JNC SKIP - If no carry, skip incrementing higher byte
- INRD - Increment register D to store carry (upper byte).
- DCR C - Decrement multiplier count
- JNZ LOOP - Repeat addition until multiplier becomes zero
- STA 2502 - Store lower byte of result
- MOV A, D - Move upper byte (carry count) to accumulator

STA 2503 - Store upper byte of result
HLT - Stop program execution

b) Bit rotation method

Memory

2500H - Multiplicand

2501H - Multiplier

2502H - Lower byte of result

2503H - Upper byte of result

Program Explanation

- LXI H, 2500 - Load HL with address of multiplicand
- MOV E, M - Load multiplicand into register E
- MVI D, 00 - Clear registers (higher byte)
- INX H - Move to multiplier address
- MOV A, M - Load multiplier into accumulator
- MVI C, 08 - Initialize loop counter for 8 bits
- LXI H, 0000 - Clear HL pair to store result
- DAD H - Left shift parallel result (HEHLX2)
- RAL - Rotate accumulator left through carry
- JNC LOOP2 - If Carry is zero, skip addition
- DAD D - Add multiplicand to result
- DCR C - Decrement bit counter
- JNZ LOOP1 - Repeat until all 8 bits are processed
- SHLD 2502 - Store Result (L → 2502H, H → 2503H)
- HLT - Stop program execution.



Results:

→

Program 1

Multiplicand = 05

Multiplier = 03

Result = 0F

→

Program 2

Multiplicand = 04

Multiplier = 04

Result = 10

Name : Divyansh Tripathi

Roll no : 123EE0344